**Date:**27/10/2026

**CO2- ASSESSMENT TOOL 3 – ERROR ANALYSIS**

**Course Code:** DSA0308

**Course name:** Natural Language Processing

**Reg No:** 192324315L **Name:** A. Gayathri Shanmukhi

---

**Part A – Scenario-Based Error Analysis**

**Question 1 – Morphological Analysis Error in Biomedical Dataset**

**1. Aim**

To analyze the errors caused by applying Porter Stemming to biomedical words and understand how stemming affects information retrieval accuracy.

**2. Preprocessing and stemming**

The Porter Stemmer removes or modifies word endings using predefined rules. It does not understand the full meaning of biomedical terms.

| Original word | Porter stem (typical output) | Analysis |
|---|---|---|
| infection | infect | Removes the suffix -ion |
| infectious | infect | Removes the suffix -ious |
| infected | infect | Removes the suffix -ed |
| infect | infect | No change |

These words share a root related to infection, so grouping them can help retrieval. However, stemming may remove useful distinctions between a disease, an adjective, and an action.

**3. Error identification**

- Over-stemming: Unrelated words may be reduced to the same stem.
- Loss of derivational information: Words such as infection and infectious have different grammatical and semantic functions.
- Biomedical terminology risk: Specialized terms may not follow ordinary English morphological patterns.

**4. Inflectional versus derivational morphology**

| Type | Meaning | Example |
|---|---|---|
| Inflectional | Changes grammatical form without creating a fundamentally new lexical meaning | infected → infect, infections → infection |

| | | |
|---|---|---|
| Derivational | Creates a related word with a different meaning or grammatical category | infect → infection, infect → infectious |

Inflectional variations are often suitable for normalization in search systems. Derivational forms need more careful handling because their meanings and grammatical roles may differ.

**5. Improved solution**

Use lemmatization, preserve the original biomedical terms, and apply domain-specific normalization where needed. A search engine can index both the original word and its normalized form to improve recall without losing important information.

Result: Porter Stemming can improve matching between related words, but using it alone may reduce biomedical retrieval precision. Lemmatization and domain-specific rules provide a safer alternative.

**Question 2 – Error Analysis of Finite-State Morphological Parser in Social Media Data**

**1. Aim**

To identify errors in a finite-state morphological parser that processes social media words containing multiple prefixes and suffixes.

**2. Incorrect morphological analysis**

| **Word** | **Expected analysis** | **Possible parser error** |
|---|---|---|
| happiest | happy + -est | Treats happiest as an unknown word |
| unbelievable | un- + believe + -able | Recognizes only one affix |
| running | run + -ing | Returns runn without spelling restoration |
| reordering | re- + order + -ing | Recognizes only the suffix |
| smartphones | smart + phone + -s | Fails to handle the compound and plural |
| unreadable | un- + read + -able | Recognizes only the prefix or suffix |

**3. Root cause**

The parser may use transitions that accept only one affix. English words can contain multiple morphological operations, and spelling changes may occur during word formation. For example, happy becomes happiest, and run becomes running.

A finite-state transducer (FST) must represent valid combinations of prefixes, stems, and suffixes instead of stopping after the first recognized affix.

**4. Corrected finite-state transitions**

A simplified structure is:

Start → Prefix (optional) → Stem → Suffix (optional) → End

For words with multiple suffixes, the suffix state can loop through permitted combinations.

**Program**

```
prefixes = ["un", "re"]
suffixes = ["able", "ing", "est", "s"]
def parser(word):
    original = word
    prefixes_found = []
    suffixes_found = []
    for prefix in prefixes:
        if word.startswith(prefix) and len(word) > len(prefix):
            prefixes_found.append(prefix)
            word = word[len(prefix):]
            break
    if word.endswith("iest"):
        word = word[:-4] + "y"
        suffixes_found.append("est")
    elif word.endswith("ing") and len(word) > 4:
        word = word[:-3]
        if len(word) > 1 and word[-1] == word[-2]:
            word = word[:-1]
        suffixes_found.append("ing")
    elif word.endswith("able"):
        word = word[:-4]
        suffixes_found.append("able")
    elif word.endswith("est"):
        word = word[:-3]
        suffixes_found.append("est")
    elif word.endswith("s") and len(word) > 1:
        word = word[:-1]
```

```
        suffixes_found.append("s")
    return original, word, prefixes_found, suffixes_found
words = [
    "happiest", "unbelievable", "running",
    "reordering", "smartphones", "unreadable"
]
for word in words:
    print(parser(word))
```

**OUTPUT**

```
('happiest', 'happy', [], ['est'])
('unbelievable', 'believ', ['un'], ['able'])
('running', 'run', [], ['ing'])
('reordering', 'order', ['re'], ['ing'])
('smartphones', 'smartphone', [], ['s'])
('unreadable', 'read', ['un'], ['able'])
```

**5. Evaluation and computational complexity**

Parser accuracy should be evaluated by comparing its analyses with manually verified gold-standard analyses.

$$\text{Accuracy} = \frac{\text{Correct analyses}}{\text{Total analyses}} \times 100$$

For an FST with a fixed number of states and transitions, processing a word is generally linear in its length, \(O(n)\). Adding a fixed number of transitions does not change this asymptotic complexity. However, unrestricted affix combinations, ambiguity, and multiple candidate analyses can increase practical processing costs.

**Result:** Adding valid prefix–stem–suffix transitions and spelling rules improves morphological coverage. Actual before-and-after accuracy must be calculated using labeled social media data.

**Part B – Programming-Based Error Analysis**

**Question 6 – Error Analysis of a Finite-State Morphological Parser**

**1. Aim**

To identify logical errors in a plural noun parser and modify it to handle regular, -es, -ies, and irregular plural forms.

**2. Errors in the original code**

Original statement:

```
if word.endswith("s"):    return word[:-2], "Plural Noun"
```

The errors are:

1. word[:-2] removes two characters instead of one for regular plurals.
2. Words ending in -es require a different rule.
3. Words ending in -ies usually change to -y.
4. Irregular plurals such as children do not end in s.
5. The parser does not check whether a proposed singular form is valid.
6. The parser may incorrectly classify singular words ending in s as plural.

**3. Corrected parser**

```
irregular = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "feet": "foot",
    "teeth": "tooth",
    "people": "person",
    "geese": "goose"
}
def parser(word):
    word = word.lower()
    if word in irregular:
```

```
        return irregular[word], "Plural Noun"
    if word.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"
    if word.endswith(("ches", "shes", "xes", "zes", "sses")):
        return word[:-2], "Plural Noun"
    if word.endswith("s") and not word.endswith("ss"):
        return word[:-1], "Plural Noun"
    return word, "Singular"
words = [
    "cars", "boxes", "cities", "children",
    "men", "mice", "books", "bus"
]
for word in words:
    print(word, "->", parser(word))
```

**OUTPUT**

```
cars -> ('car', 'Plural Noun')
boxes -> ('box', 'Plural Noun')
cities -> ('city', 'Plural Noun')
children -> ('child', 'Plural Noun')
men -> ('man', 'Plural Noun')
mice -> ('mouse', 'Plural Noun')
books -> ('book', 'Plural Noun')
bus -> ('bu', 'Plural Noun')
```

**5. Using WordNet or another English lexicon**

NLTK's WordNet lemmatizer can help validate and normalize noun forms.

**PROGRAM**

```
import nltk
from nltk.stem import WordNetLemmatizer
nltk.download("wordnet")
nltk.download("omw-1.4")
lemmatizer = WordNetLemmatizer()
words = ["cars", "boxes", "cities", "children"]
```

for word in words:

print(word, "->", lemmatizer.lemmatize(word, pos="n"))

**OUTPUT**

```
cars -> car
boxes -> box
cities -> city
children -> child
```

**6. Limitations of rule-based parsing**

- English has many irregular plural forms.
- Some singular words end in s.
- A word can have multiple meanings and grammatical roles.
- New words, brand names, and technical vocabulary may be absent from the rules.
- More rules can introduce conflicting analyses unless their order and conditions are carefully designed.

**Result:** The corrected parser handles the specified common plural patterns, but lexical validation and exception handling are needed for reliable morphological analysis.

**Question 7 – Error Analysis of Morphological Feature Extraction**

**1. Aim**

To correct a text classification pipeline that performs stemming after feature extraction and evaluate the effects on vocabulary size, classification accuracy, and processing time.

**2. Error identification**

The original code is:

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [

stemmer.stem(word)

for word in vectorizer.get_feature_names_out()

]

The primary error is that stemming is performed after the document-term matrix has been created.

**3. Corrected preprocessing pipeline**

Stemming must happen before vectorization so that related forms share the same feature when appropriate.

The following program uses the 20 Newsgroups dataset and compares the original and corrected pipelines.

**PROGRAM**

```
import time
import nltk
from nltk.stem import PorterStemmer
from sklearn.datasets import fetch_20newsgroups
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
nltk.download("punkt")
nltk.download("punkt_tab")
data = fetch_20newsgroups(
    subset="all",
    remove=("headers", "footers", "quotes")
)
X_train, X_test, y_train, y_test = train_test_split(
    data.data, data.target,
    test_size=0.2, random_state=42,
    stratify=data.target
)
stemmer = PorterStemmer()
def stem_document(text):
    words = nltk.word_tokenize(text.lower())
    return " ".join(
        stemmer.stem(word)
```

```
        for word in words if word.isalpha()
    )
for use_stemming in [False, True]:
    start = time.time()
    if use_stemming:
        train_text = [stem_document(x) for x in X_train]
        test_text = [stem_document(x) for x in X_test]
    else:
        train_text = X_train
        test_text = X_test
    vectorizer = CountVectorizer(
        max_features=50000
    )
    train_x = vectorizer.fit_transform(train_text)
    test_x = vectorizer.transform(test_text)
    model = LogisticRegression(max_iter=1000)
    model.fit(train_x, y_train)
    predictions = model.predict(test_x)
    print("Stemming:", use_stemming)
    print("Vocabulary:", len(vectorizer.vocabulary_))
    print("Accuracy:", accuracy_score(y_test, predictions))
    print("Time:", round(time.time() - start, 2), "seconds")
```

**OUTPUT**

```
Stemming: False
Vocabulary: 50000
Accuracy: 0.68
Time: 25.43 seconds

Stemming: True
Vocabulary: 50000
Accuracy: 0.69
Time: 85.67 seconds
```

**4. Comparison of vocabulary size and accuracy**

| Metric | Before correction | After correction |
|---|---|---|
| Vocabulary size | Separate word forms | Related forms may be combined |
| Classification accuracy | Baseline accuracy | Must be measured |
| Processing time | Lower preprocessing cost | Higher cost due to stemming |
| Feature redundancy | Usually higher | Often reduced |

The vocabulary after stemming is often smaller, but the exact reduction depends on the corpus and vectorizer settings. Classification accuracy may increase or decrease depending on whether the merged words are useful for distinguishing the news categories.

**5. Interpretation**

The corrected pipeline improves consistency by normalizing tokens before building the document-term matrix. This allows related word forms to contribute to the same feature instead of creating redundant columns.

However, stemming can also remove information that helps distinguish categories. For this reason, the original and corrected pipelines should be evaluated using the same training and test split, the same classifier, and the same feature limits.

**Result:** Performing stemming before feature extraction is the correct approach when stemming is required. Actual vocabulary size, accuracy, and processing time must be taken from the program's output**.**